Explain the Linux file system hierarchy (/etc, /var/log, /home, /tmp) and file permissions: chmod, chown and SUID with examples.

Linux is an operating system that organizes files and directories in a hierarchical structure. Unlike Windows, which commonly uses drive letters, Linux uses a single directory tree that begins at the root directory /.

Understanding the Linux file system and permissions is important for system administration and cybersecurity because it helps users manage files, control access, and identify potential security weaknesses.

Linux File System Hierarchy

The following directories are commonly used in Linux:

Directory	Purpose	Example
/	The root of the entire file system. All directories exist under it.	/home/user
/etc	Stores system-wide configuration files.	/etc/ssh/sshd_config
/var/log	Stores system and application log files.	/var/log/syslog
/home	Contains personal directories for regular users.	/home/student
/tmp	Stores temporary files that may be deleted automatically.	/tmp/test.txt
/bin	Contains essential user commands; on many modern systems, it links to /usr/bin.	/bin/ls
/usr	Contains many user applications, libraries, and read-only program data.	/usr/bin/grep
/root	The home directory of the root administrator.	/root/
/dev	Contains device files used to interact with hardware and virtual devices.	/dev/null
/var	Stores variable data such as logs, caches, and service data.	/var/cache

Important directories

- **/etc:** Administrators use this directory to configure system services and applications.
- **/var/log:** Security professionals examine logs here to investigate errors, authentication events, and suspicious activity. Log locations vary by Linux distribution.
- **/home:** Stores users' personal files and settings.
- **/tmp:** Holds temporary files. Sensitive data should not be stored here without appropriate protections.

Linux File Permissions

Linux permissions determine who can read, modify, or execute a file and who can access a directory.

There are three permission categories:

- **User (u):** The owner of the file.
- **Group (g):** Users who belong to the file's assigned group.
- **Others (o):** All other users.

There are three basic permissions:

Permission	Symbol	Numeric value	Meaning
Read	r	4	Read file contents or list directory entries.
Write	w	2	Modify a file or change directory entries, subject to other controls.
Execute	x	1	Execute a file or traverse/search a directory.

Example of permissions

Run:

ls -l report.txt

Example output:

-rw-r----- 1 alice analysts 1200 Oct 20 10:00 report.txt

This means:

- alice is the file owner and can read and write the file.
- Members of the analysts group can read it.
- Other users have no permissions on this file.

The chmod Command

chmod means **change mode**. It changes file or directory permissions.

Example 1: Numeric permissions

chmod 640 report.txt

Permission breakdown:

- Owner: 6 = read (4) + write (2).
- Group: 4 = read only.
- Others: 0 = no permissions.

The resulting permissions are rw-r-----.

Example 2: Symbolic permissions

chmod u+x script.sh

This adds execute permission for the file owner.

Other examples:

chmod g-w report.txt

chmod o-r secret.txt

- chmod g-w removes write permission from the group.
- chmod o-r removes read permission from others.

Use chmod to grant only the permissions required for a task. Avoid giving everyone write access to sensitive files.

The chown Command

chown means **change owner**. It changes the owner of a file or directory and can also change its group.

Example 1: Change the owner

sudo chown alice report.txt

This changes the file owner to alice.

Example 2: Change owner and group

sudo chown alice:analysts report.txt

This assigns alice as the owner and analysts as the group.

Verify ownership

ls -l report.txt

Use chown when files need to belong to the correct user or group. Administrative privileges may be required.

What Is SUID?

SUID (Set User ID) is a special Linux permission bit used mainly on executable files. When a SUID executable runs, it normally uses the file owner's effective user ID rather than the launching user's effective user ID.

For example, certain system utilities use SUID to perform a limited privileged operation on behalf of a regular user.

Example: Identify SUID files

find / -type f -perm -4000 2>/dev/null

This searches for regular files with the SUID bit set. Access restrictions may prevent the command from examining every directory.

Example: Recognize SUID permissions

-rwsr-xr-x

The s in the owner's execute position indicates SUID is set and the owner-execute bit is enabled.

Why can SUID be a security risk?

- A vulnerable SUID program may allow unintended privileged actions.
- Unnecessary SUID permissions increase the potential for privilege escalation.
- Poorly designed privileged programs may expose sensitive files or system functions.

Security best practice: Audit SUID programs, keep them updated, and avoid setting SUID on arbitrary executables. Only test permission changes on systems you own or are authorized to administer.

Useful Commands for File and Permission Management

Command	Purpose
pwd	Displays the current directory.
ls -l	Displays files with permissions and ownership.
cd /etc	Changes to the /etc directory.
find /home -type f	Finds regular files under /home.

chmod 640 file.txt	Sets owner read/write and group read permissions.
chown user:group file.txt	Changes file ownership and group.
id	Displays the current user's identity and groups.
umask	Displays or sets the default permission mask.

Conclusion

The Linux file system organizes system files, configuration, logs, and user data into a structured directory hierarchy.

File permissions control access through read, write, and execute rights. The chmod command changes permissions, chown changes ownership, and SUID allows certain executables to run with the file owner's effective user ID.

Understanding these concepts is essential for managing Linux systems securely and recognizing possible privilege-escalation risks.